

WEEK6 Transaction Properties

Transactions — The ACID Properties

Study notes based on the Stanford lecture video on transaction properties (Widom). This video assumes a prior lecture introduced **transactions** as the solution to two problems: **concurrency control** (multiple clients accessing the database at once) and **system failure** (crashes, power loss). This note covers the **four formal properties** every transaction system guarantees — the **ACID** properties.

1. What Is a Transaction? (Recap)

A **transaction** is a **sequence of operations treated as a single unit**.

Two core guarantees motivate the whole concept:

1. **Concurrency illusion:** Even with many clients hitting the database simultaneously, each transaction *appears* to run in **isolation** — as if it had the database to itself.
2. **Failure safety:** If the system crashes (software bug, hardware failure, power loss), a transaction's changes are reflected **entirely or not at all** — never partially.

These two guarantees, formalized, give us **ACID**:

Letter	Property	One-line meaning
A	Atomicity	All-or-nothing execution
C	Consistency	Transactions preserve integrity constraints
I	Isolation	Concurrent transactions behave as if run one-at-a-time
D	Durability	Once committed, changes survive

Letter	Property	One-line meaning
		crashes
The lecture covers these out of alphabetical order — Isolation, then Durability, then Atomicity, then Consistency — because that's the order that builds intuition most naturally. These notes follow the same order.		

2. Isolation — The "I" in ACID

2.1 The setup

Picture multiple **clients**, each issuing a **sequence of transactions** to the database:

Client 1: T1 → T2 → T3 → ...

Client 2: T9 → T10 → T11 → ...

Each transaction is itself a sequence of statements:

T1: statement1 → statement2 → statement3 → ...

The database may **interleave** statements from different clients' transactions for performance — but isolation demands that the *observable result* never reveals that interleaving.

2.2 The formal definition: Serializability

Serializability: Operations from different transactions may be physically interleaved during execution, but the *result* must be **equivalent to some serial (sequential, non-interleaved) ordering** of those same transactions.

In other words: the system can run things concurrently under the hood, but it must always be possible to find **some** order — T1, T2, T9, T10, T3, ... for example — such that running the transactions one-at-a-time in that order would produce the **same final database state**.

Key nuance: Serializability guarantees *some* valid order exists — it does **not** guarantee any *particular* order. If your application needs T1 to happen before T2 specifically, that ordering must be **enforced by the application**, not assumed from isolation alone.

2.3 How is this even possible?

The database uses **locking protocols** to interleave operations safely while preserving the serializability guarantee.

Implementation is **out of scope** for this course — what matters from the application/user perspective is **the guarantee itself**, not how the DBMS achieves it.

2.4 Worked examples — applying serializability

These four scenarios revisit the concurrency problems from the prior lecture, now solved by wrapping each client's operation in a transaction.

Example A — Two clients update Stanford's enrollment

Both clients add to the same enrollment value (e.g., +1000 and +1500).

Possible serial order	Result
T1 then T2	Enrollment goes 15,000 → (15,000+1000) → (16,000+1500) = 17,500
T2 then T1	Enrollment goes 15,000 → (15,000+1500) → (16,500+1000) = 17,500

Either order gives the correct final answer (17,500) — this update is *commutative*, so serializability alone fully solves the original lost-update problem.

Example B — Two clients modify different columns of the same row

Client 1 changes student 123's major in Apply; Client 2 changes the decision for the same row.

Possible serial order	Result
T1 then T2	Both the major change and the decision change are reflected
T2 then T1	Both changes are reflected

Both orders are correct — no change is lost, unlike the un-transacted interleaved version from the earlier lecture where one update could silently overwrite the other.

Example C — Modifying Apply based on Student.GPA while simultaneously updating GPA

Client 1 reads GPA to decide on application decisions; Client 2 is updating the GPA values themselves.

Possible serial order	Result
T1 then T2	Decisions are set using the old GPAs; GPAs updated afterward
T2 then T1	GPAs updated first; decisions are set using the new GPAs

This is the interesting case. Both orders produce a database that's internally *consistent* — but they are **different results**, and the order genuinely matters for the application's meaning. Serializability guarantees one of these two orders happened — but **does not guarantee which one**. If the application cares (and here, it plausibly does), it must control the order explicitly (e.g., by issuing T1 and waiting for it to commit before issuing T2).

Example D — Moving rows from Apply to Archive while counting rows

Client 1 moves records from Apply to Archive; Client 2 counts the tuples in Apply.

Possible serial order	Result
T1 then T2	Count reflects zero moved records (they're already gone)
T2 then T1	Count reflects the original full set

Again — **both are valid serializable outcomes**, but they differ, and only the application knows which one it actually wants.

2.5 Takeaway on isolation

Serializability is a **strong, well-defined guarantee**: no partial/interleaved corruption is ever visible. But it is **not** a guarantee about *which* serial order occurs. **Ordering control belongs to the application**, not the isolation property.

3. Durability — The "D" in ACID

3.1 The guarantee

Now we only need **one client** to understand durability:

Client: T1 → T2 → ... → Tn → [commit]

Durability: Once a transaction **commits**, its effects are **permanent** — they will survive any subsequent system crash. When the system comes back up after a crash, all committed effects are still there.

So if a transaction commits and *then* the system crashes (for any reason — software, hardware, power), the application can be 100% confident the changes are safely in the database once it recovers.

3.2 How is this implemented?

Via **logging protocols** — the database doesn't only rely on data sitting safely in memory or even on disk in its final location; it uses a durable log that survives crashes and can be replayed/recovered from.

Again, **implementation is out of scope**. From the application perspective, the only thing that matters is: *"committed = permanent, guaranteed."*

4. Atomicity — The "A" in ACID

4.1 The guarantee

Again, consider one client and one transaction:

T2: statement1 → statement2 → statement3 → ... → [commit]

Atomicity: if a crash occurs **during** execution of a transaction (before it commits), the transaction's effects are **either fully applied or not applied at all** — never partially.

This is the **"all-or-nothing"** property. The database will never end up in a state reflecting *some but not all* of a transaction's statements.

4.2 What this means for applications

If an application submits a transaction and the system crashes mid-execution:

- The application may receive an **error** (the transaction did not complete).
- It can be **guaranteed** that **none** of that transaction's effects made it into the database.
- The application must **resubmit** / **restart** the transaction if it still wants those changes applied.

4.3 Implementation: logging + rollback

Atomicity is also implemented using a **logging mechanism**. On recovery from a crash, the system runs a process that **undoes partial effects** of any transactions that were mid-flight when the crash occurred.

4.4 Rollback / Abort — exposed to applications too

This "undo partial effects" mechanism has a name — **transaction rollback** (also called **transaction abort**) — and it isn't just an internal recovery tool. It's also an **operation applications can invoke deliberately**.

Trigger	Who initiates it
System error / crash recovery	The system , automatically
Explicit decision mid-transaction	The client/application , on purpose

4.5 Worked example — client-initiated rollback

```
BEGIN TRANSACTION;
-- ask the user for input
-- run some SQL commands based on that input (modifying the DB)
-- ask the user: "are you happy with these results?"
IF user says OK:
    COMMIT;
ELSE:
    ROLLBACK; -- automatically undoes the SQL commands above
END;
```

If the user is unhappy, ROLLBACK automatically undoes the database modifications — the application doesn't need to write explicit "undo" logic itself. This is a genuinely useful pattern.

4.6 Two important caveats

Caveat 1 — rollback only undoes database effects. ROLLBACK undoes changes to the data in the database. It does **not** undo:

- Local application **variables**.
- **External side effects** — e.g., if this transaction also dispensed cash from an ATM, rollback will not pull the cash back in!

Be careful designing transactions that mix database changes with real-world, irreversible side effects.

Caveat 2 — never hold a transaction open waiting on a slow/arbitrary external event. The example above (BEGIN, then *wait* for user input, *then* COMMIT/ROLLBACK) is a **bad practice**, flagged explicitly in the lecture.

Why it's bad: Transactions use **locking**. A long-running, open transaction may be **blocking other clients** from accessing the same portions of the database. If the user walks away for a coffee break — or a week — the database stays locked the whole time.

Rule of thumb: Design transactions so they are guaranteed to **run to completion quickly**. Don't pause mid-transaction waiting on unpredictable external input (user interaction, slow network calls, etc.).

5. Consistency — The "C" in ACID

5.1 The guarantee

Consistency describes how transactions interact with the database's **integrity constraints** (recall: integrity constraints define which database *states* are legal).

The deal is simple and symmetric:

- Every client may **assume** the database satisfies all integrity constraints **when its transaction begins**.
- Every client **must guarantee** that all constraints hold again **when its transaction ends** (this is typically enforced automatically by the constraint-checking subsystem covered in the constraints lecture).

5.2 Why serializability makes this work for everyone

Because of **isolation/serializability** (§2), the overall execution is equivalent to *some* sequential order of all transactions. That means:

1. The very first transaction starts with constraints holding (by assumption on the initial database state).

2. It's guaranteed to leave constraints holding when it ends (by the consistency guarantee).
3. So the *next* transaction in the serial order can, in turn, assume constraints hold at its start.
4. And so on, transitively, down the entire serial order.

Consistency is essentially "inherited" along the serial order that isolation guarantees exists — this is exactly why ACID's four letters work together as a package rather than as four independent, unrelated guarantees.

6. Summary

Property	Plain-English guarantee	Scope needed to understand it	Implementation mechanism (FYI only)
Isolation	Concurrent execution behaves as if some serial order of transactions occurred	Multiple clients	Locking protocols
Durability	Once committed, changes survive any future crash	One client, post-commit	Logging
Atomicity	A transaction's effects are all-or-nothing, even across a crash	One client, pre-commit	Logging + rollback/recovery
Consistency	Integrity constraints hold at the start and end of every	One client + the constraint system	Constraint enforcement subsystem

Property	Plain-English guarantee	Scope needed to understand it	Implementation mechanism (FYI only)
	transaction, and (via isolation) transitively throughout the serial order		

Key takeaways

- Transactions solve **two** problems at once: **concurrency control** and **system failure recovery**.
- **Isolation** $\neq$ **a specific order**. It only guarantees *some* serial-equivalent order exists. If your application logic depends on a specific ordering between transactions, **you must enforce that yourself**.
- **Rollback/abort** is both an internal recovery tool *and* a feature applications can use deliberately — but it **only undoes database state**, never external side effects or local variables.
- **Never hold a transaction open while waiting on unpredictable external input** — it can lock out other clients indefinitely.
- **Consistency rides on isolation**: because the system guarantees a valid serial order exists, and each transaction individually preserves constraints, the *whole sequence* of transactions preserves constraints throughout.

Quick reference

ACID

A — Atomicity → all-or-nothing, even across crashes

C — Consistency → constraints hold at txn start & end (relies on Isolation)

I — Isolation → behavior $\equiv$ some serial order of transactions (serializability)

D — Durability → committed = permanent, survives crashes

*Next video: a closer look at the **isolation** property — including cases where applications may want to deliberately **relax** isolation guarantees for performance, while still keeping behavior acceptable for their specific use case.*